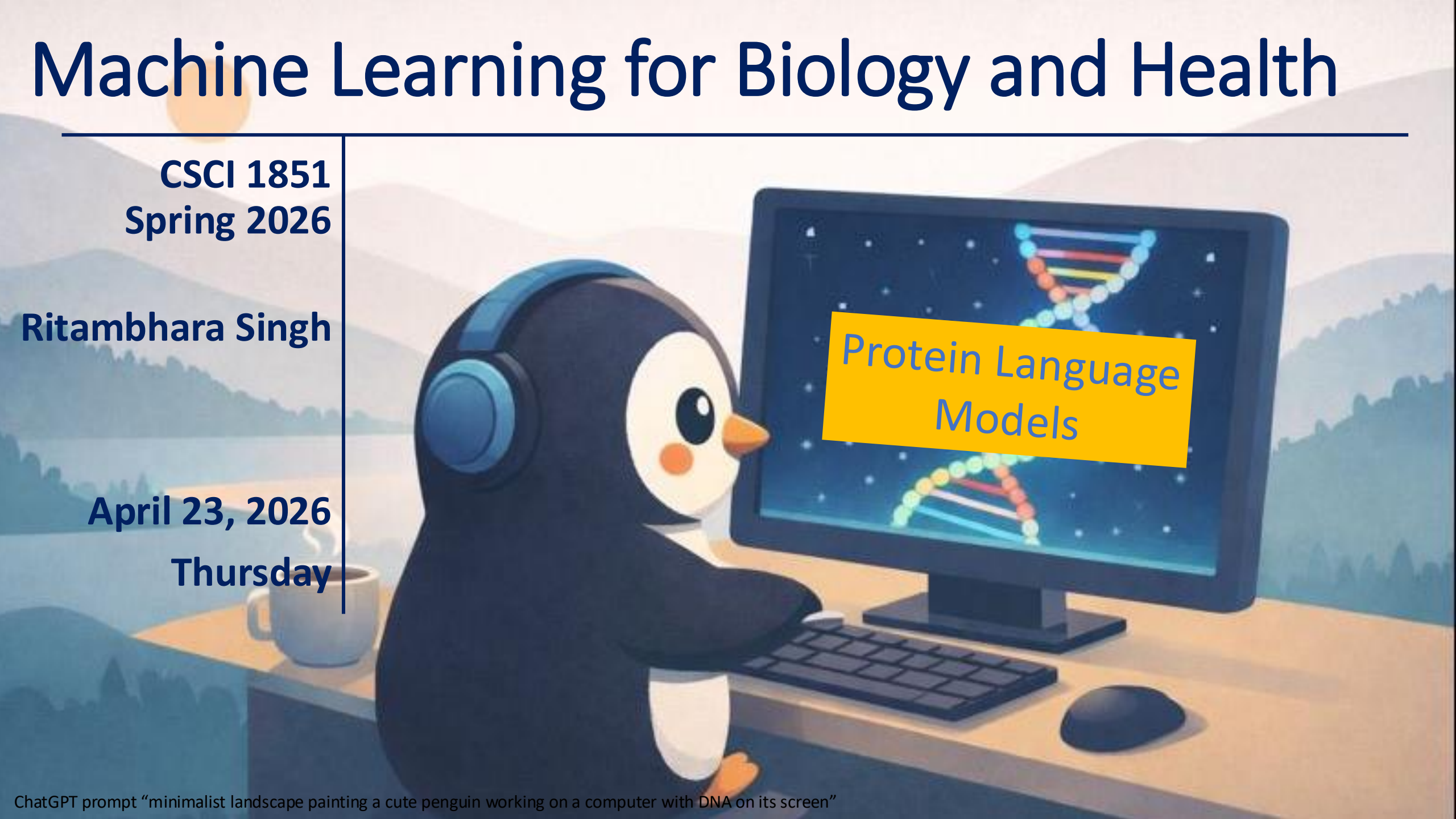
Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

April 23, 2026

Thursday



ChatGPT prompt “minimalist landscape painting a cute penguin working on a computer with DNA on its screen”

Today’s goal – learn about protein language models and how to use them

(1) Protein Language Models

(2) Hugging Face

(3) LoRA fine-tuning

(3) Class activity: ESM model for protein sequences

Foundation Models (FMs) for generalizing across tasks

Foundation models are **large** neural networks.

Trained on **large amounts** of **unlabeled** data (self-supervised learning).

Unlike more narrow traditional machine learning models, foundation models can be **easily adapted to many different tasks** without new training data.

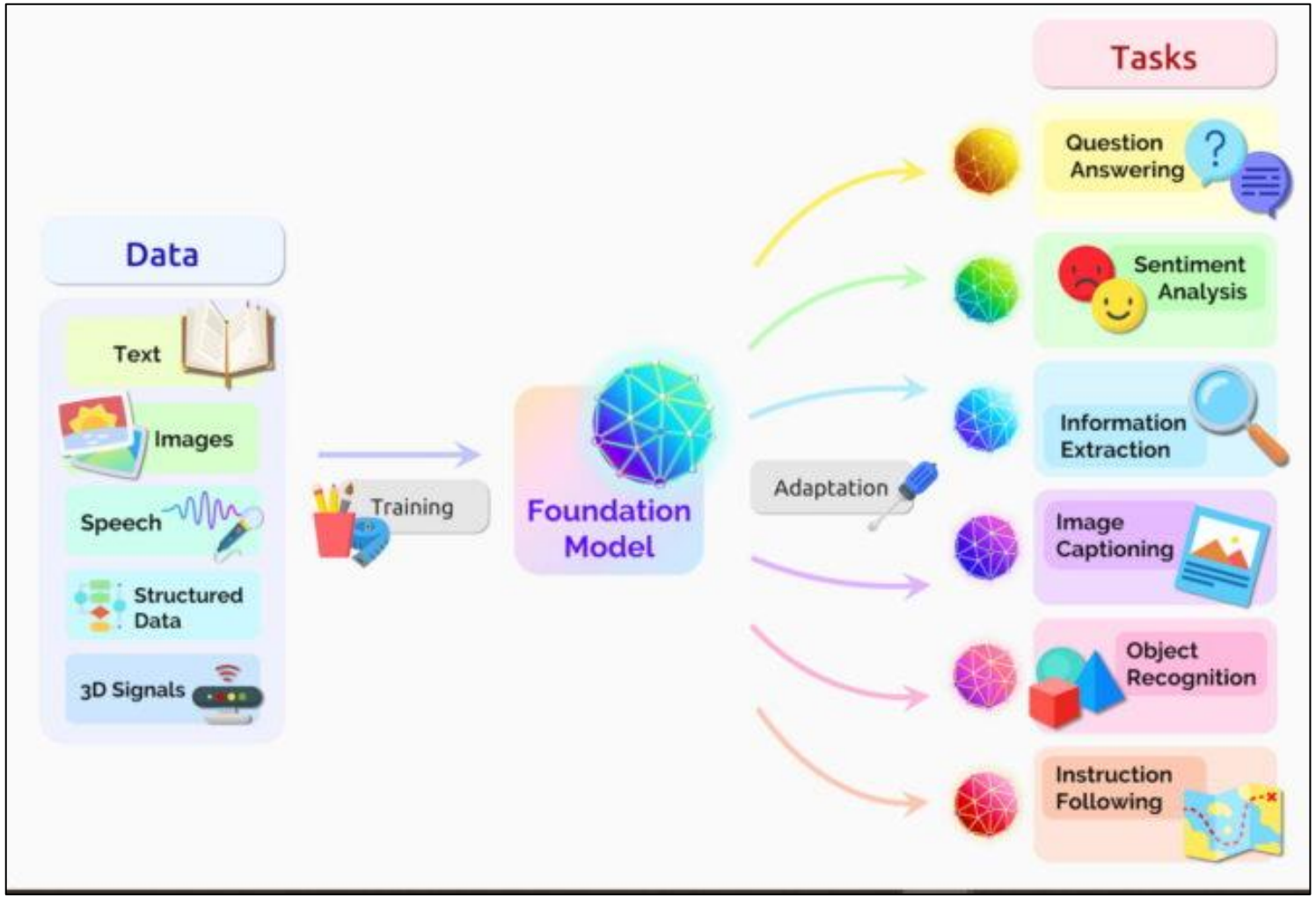
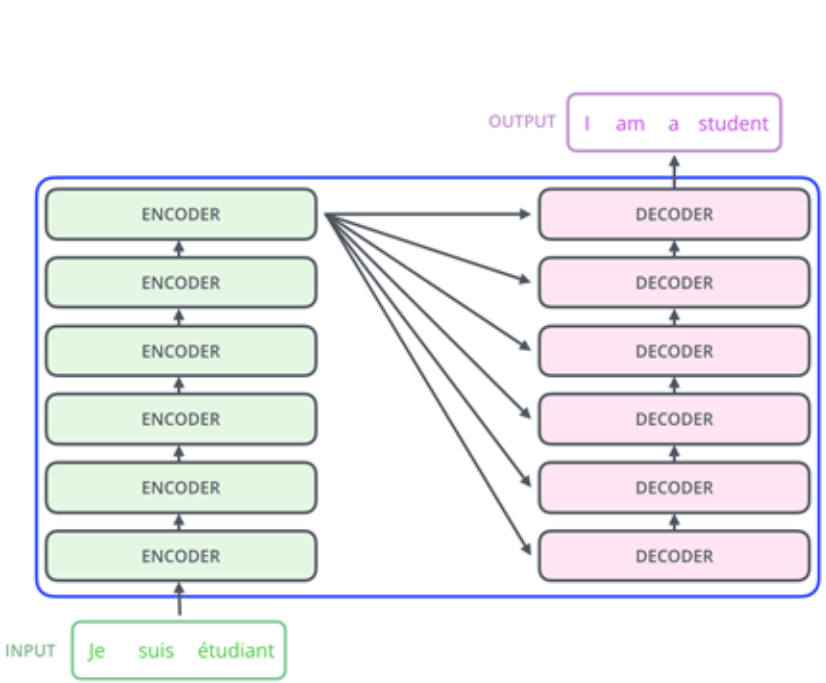
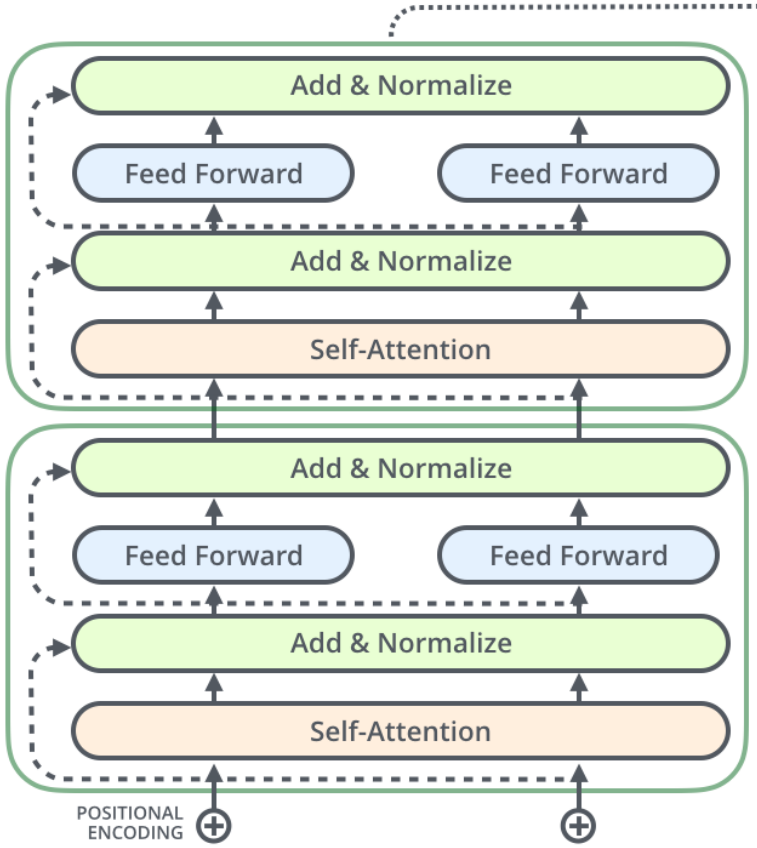
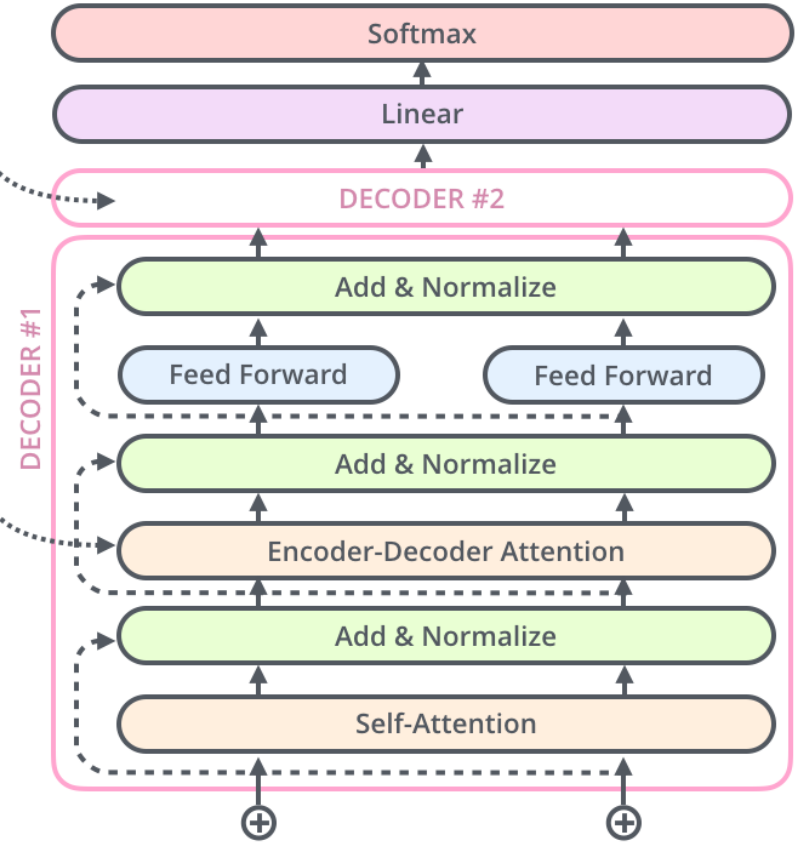


Image courtesy: https://blogs.nvidia.com/blog/what-are-foundation-models/

Encoders and Decoders Together







Alammar, Jay (2018). The Illustrated Transformer [Blog post]. Retrieved from https://jalammar.github.io/illustrated-transformer/

Foundation Model Architectures

Encoder-Only

Bidirectional Attention

Sees entire input sequence at once

Training: Masked language modeling (MLM)

Examples: BERT, RoBERTa

Best For:

- Classification tasks
- Named entity recognition
- Question answering
- Embeddings & similarity

Decoder-Only

Causal/Autoregressive

Only sees previous tokens (left-to-right)

Training: Next-token prediction

Examples: GPT, Claude, LLaMA

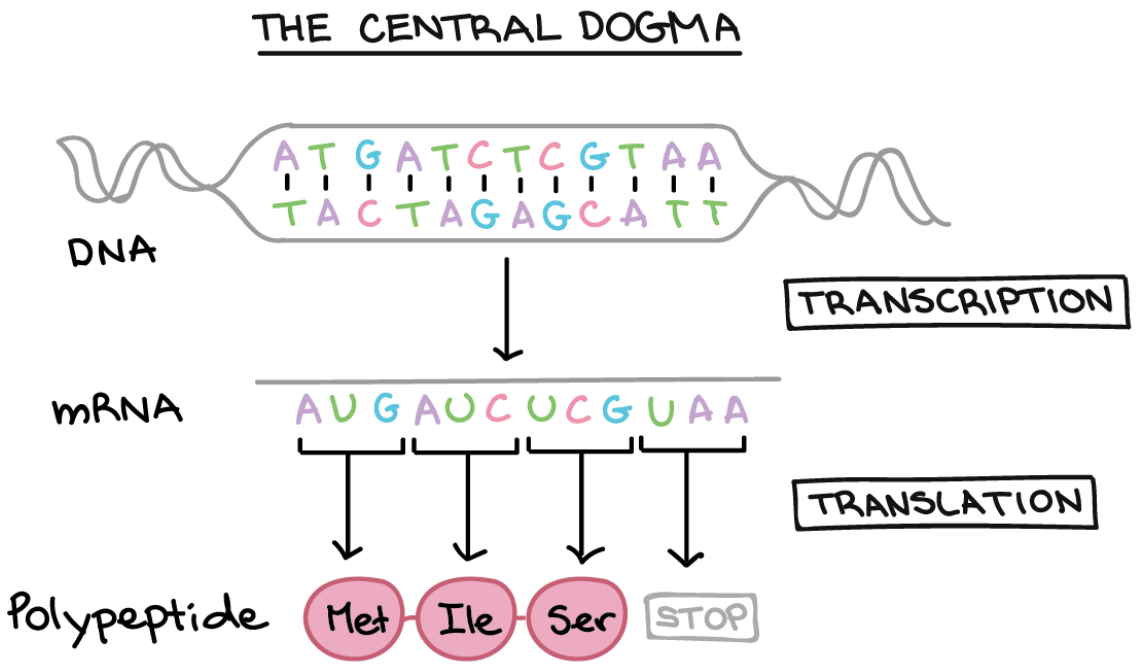
Best For:

- Text generation
- Instruction following
- Dialogue & conversation
- Code generation

Note: Decoder-only models have become dominant for large foundation models due to better scalability

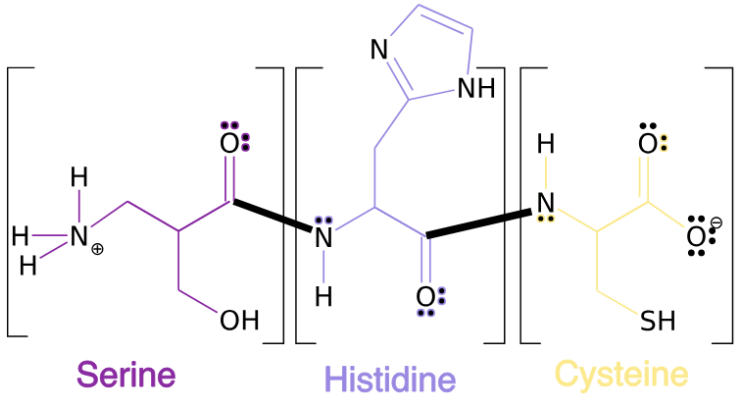
Protein Language Models

Proteins

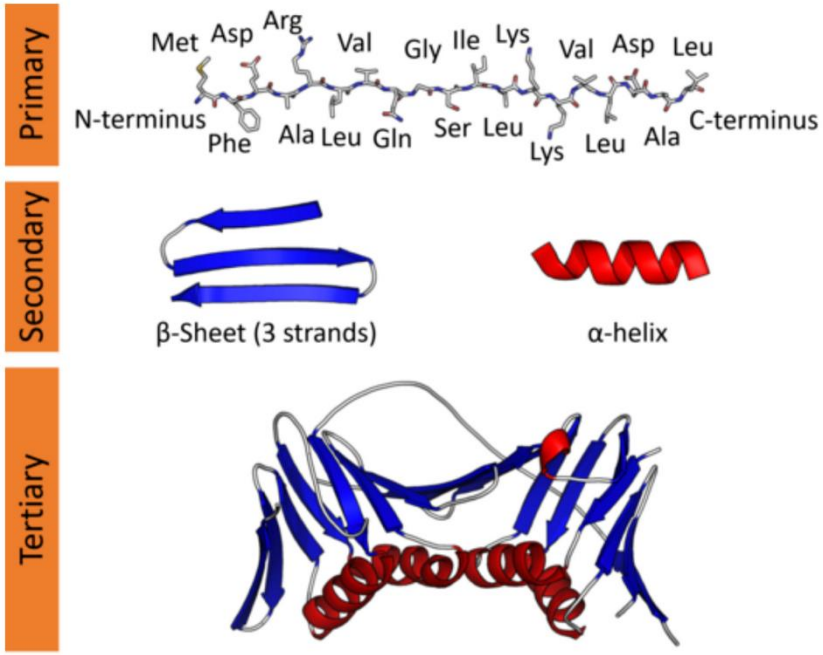


A protein sequence is made up of amino acids.

There are 20 amino acids



→ SHC



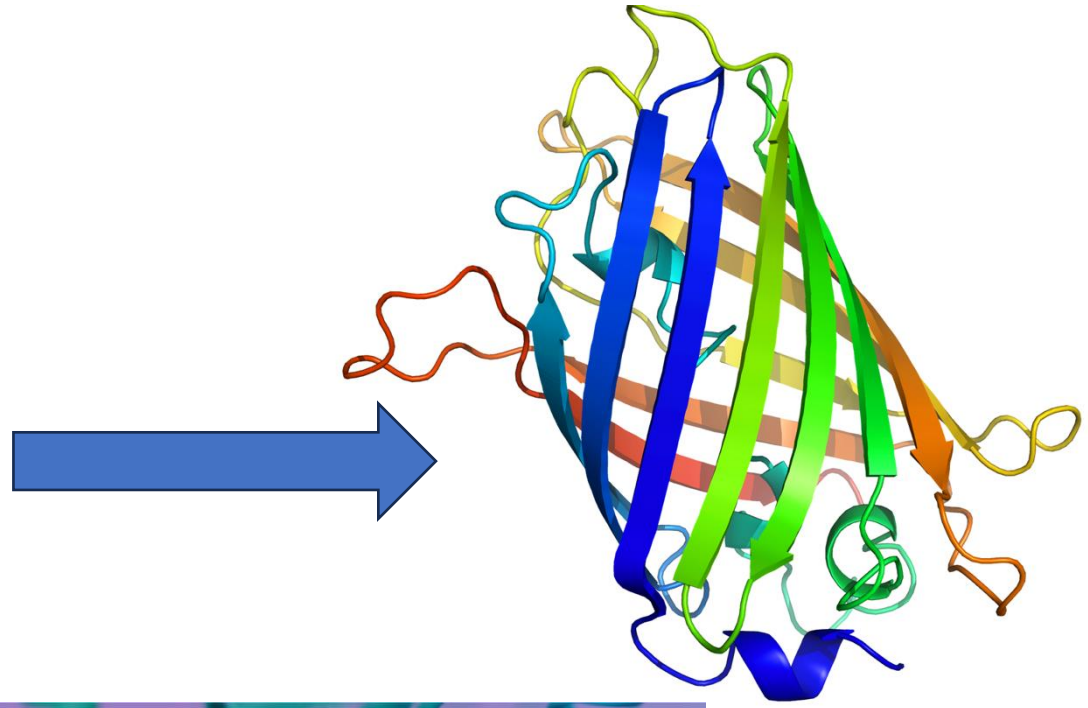
https://bair.berkeley.edu/blog/2019/11/04/proteins/

Protein prediction tasks

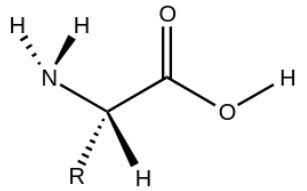
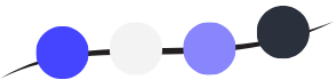
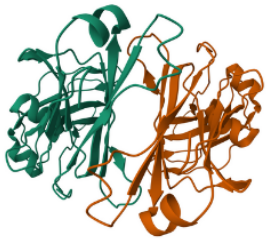
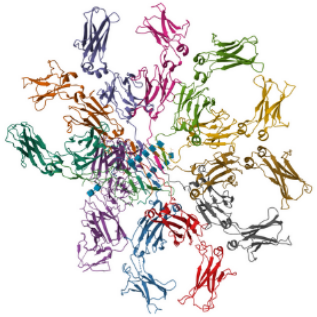
What type of learning?

What type of prediction task?

```
MSKGEELFTG VVPILVELDG DVNGHKFSVS GEGEGDATYG KLTLKFICTT
GKLPVPWPTL VTTFSYGVQC FSRYPDHMKQ HDFFKSAMPE GYVQERTIFF
KDDGNYKTRA EVKFEGDTLV NRIELKGIDF KEDGNILGHK LEYNYNSHNV
YIMADKQKNG IKVNFKIRHN IEDGSVQLAD HYQQNTPIGD GPVLLPDNHY
LSTQSALSKD PNEKRDHMVL LEFVTAAGIT HGMDELYK
```



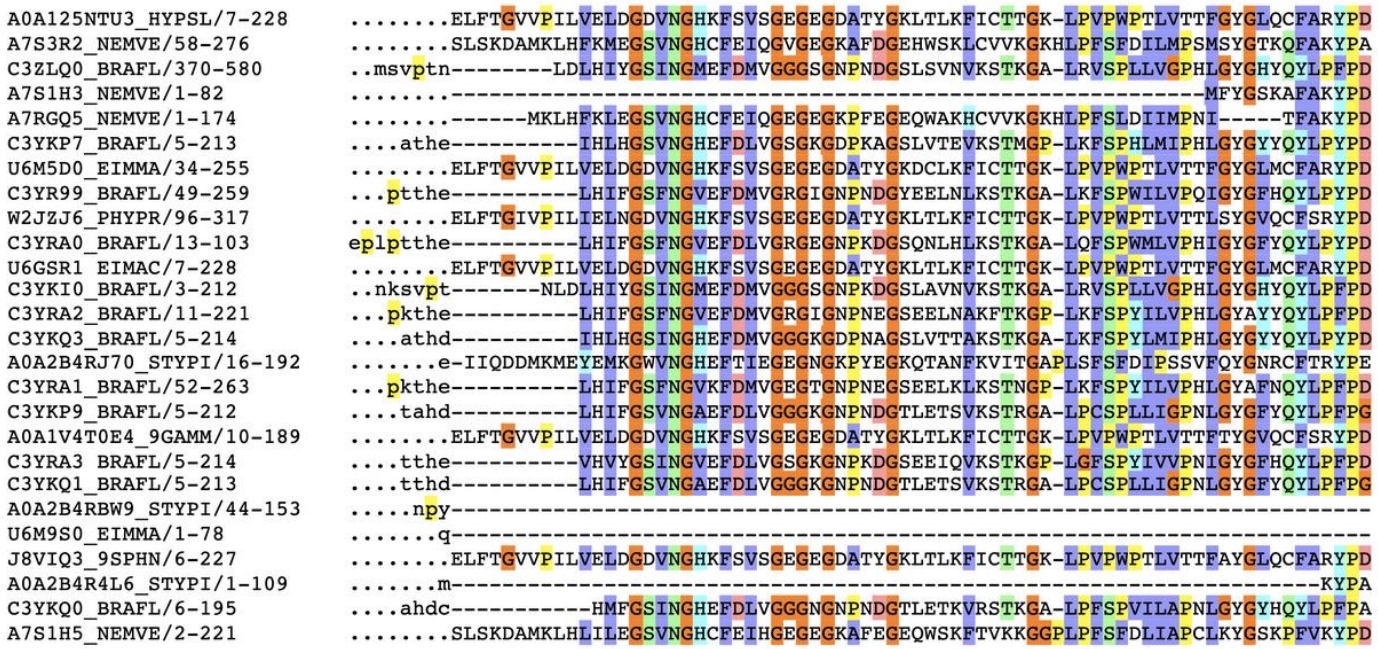
Any famous model comes to mind?

Natural languages e.g. English		Proteins e.g. IgM antibody
Letters ABCDEFGHIJK[...]z (total: 26)	**Level 1** **Alphabet**	**Amino acids** ACDEFGHIKLM[...]Y (total: 20) 
Word Reading 	**Level 2** **single-unit** **sequence**	**Protein sequence** QVQLVQSGAEVKKPG[...] 
Sentence Darcy enjoys reading a lot	**Level 3** **structural** **context**	**Single-domain 3D structure** IgM monomer Y-shaped molecule with two binding sites 
Paragraph In addition to horse riding and nature walking, Darcy enjoys reading a lot. It has always been one of his favourite activities [...]	**Level 4** **The bigger** **picture**	**Multi-domain 3D structure** IgM pentamer star-shaped molecule with ten antigen binding sites 

Complexity (+)

https://pipebio.com/blog/protein-language-models

Protein sequence datasets (>100 million sequences)



Sequential, discrete structure. Proteins are linear chains of amino acids drawn from a fixed "alphabet" of ~20 standard residues, much like sentences are sequences of tokens from a vocabulary.

Context-dependent meaning. Just as the meaning of a word depends on surrounding words, the functional role of an amino acid depends on its sequence context. A leucine in a hydrophobic core behaves very differently from one on a surface loop. Attention mechanisms are well-suited to capturing these long-range dependencies.

Conserved "grammar." Evolution imposes statistical regularities on protein sequences — certain residue combinations, motifs, and covariation patterns recur because they're structurally or functionally important.

Transfer learning works. Because the "grammar" of proteins is shared across protein families, representations learned on large diverse sequence databases transfer well to downstream tasks (structure prediction, function annotation, fitness prediction, localization), mirroring how GPT/BERT embeddings transfer across NLP tasks.

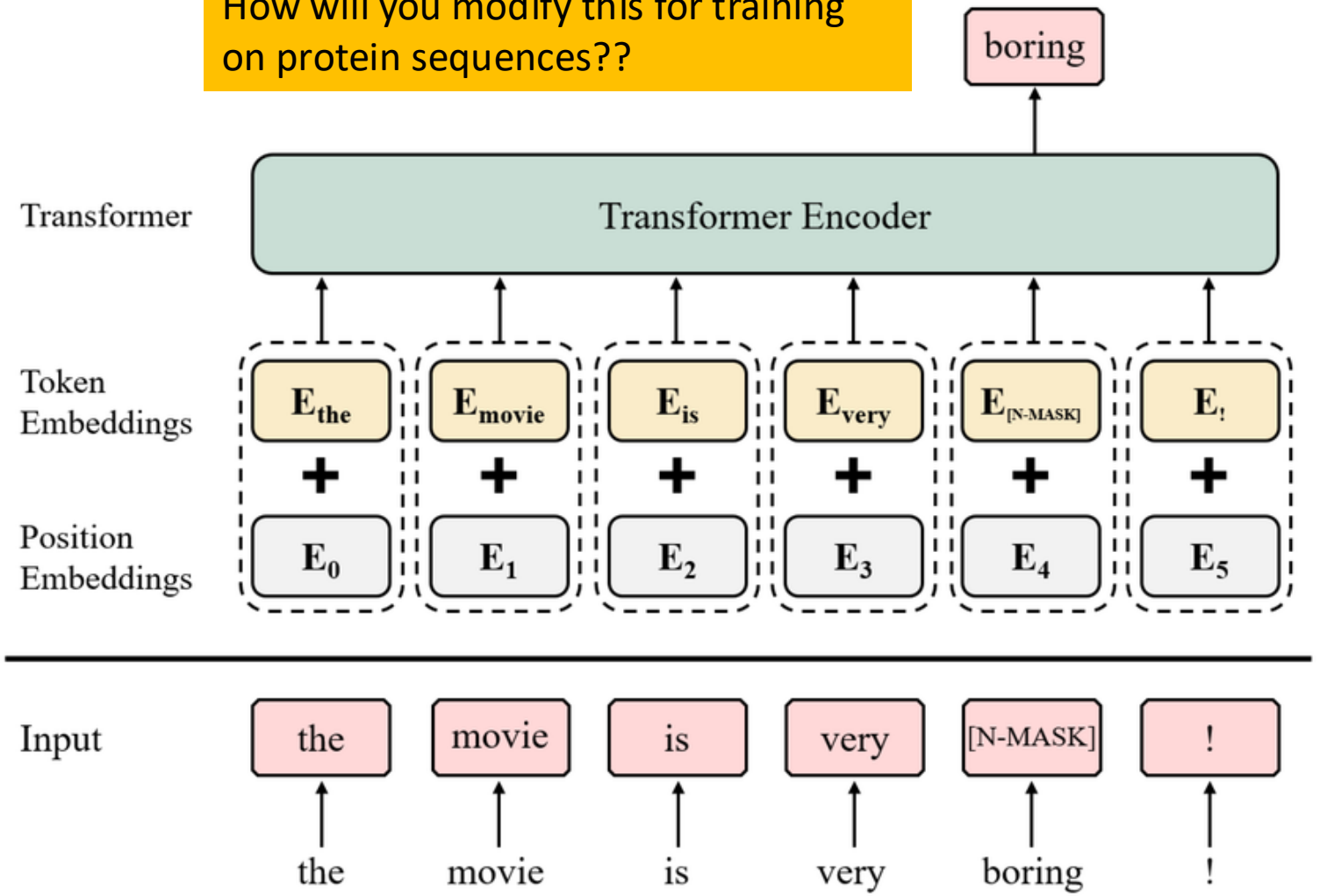
FM architecture and training for language models

Foundation models are **large** neural networks.

Trained on **large amounts** of **unlabeled** data (self-supervised learning).

Unlike more narrow traditional machine learning models, foundation models can be **easily adapted to many different tasks** without new training data.

How will you modify this for training on protein sequences??



RESEARCH ARTICLE | COMPUTER SCIENCES |

Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences

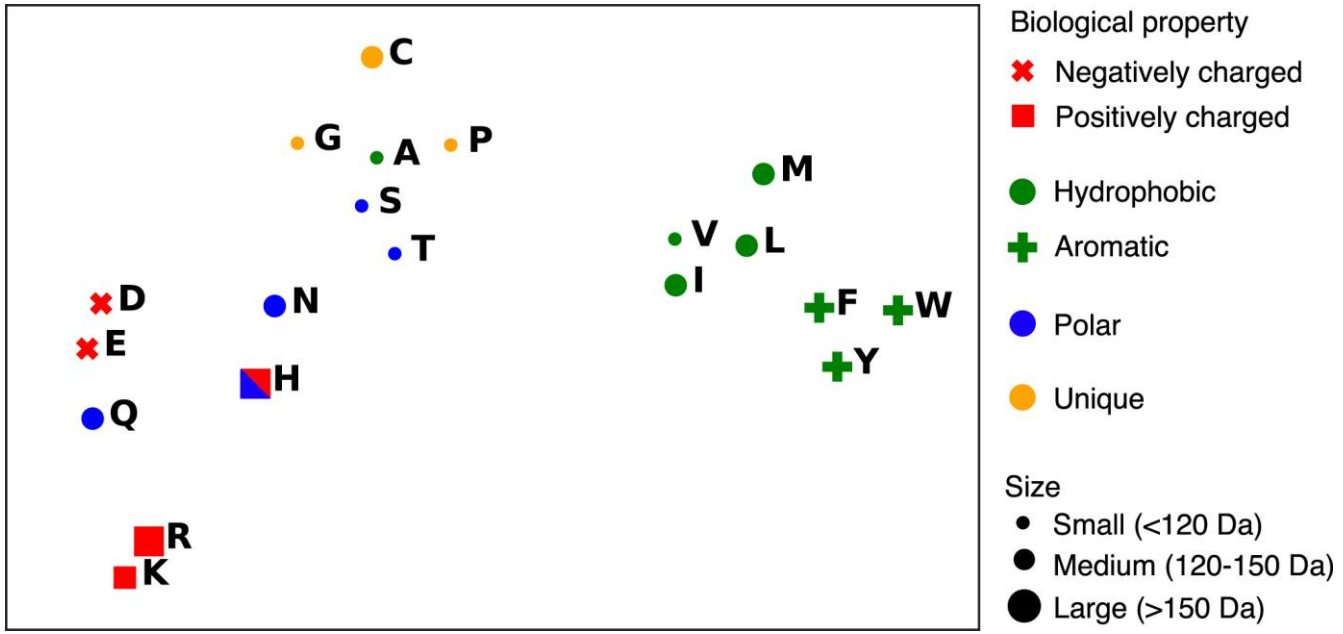
Alexander Rives, Joshua Meier, Tom Sercu, +7, and Rob Fergus Authors Info & Affiliations

Edited by David T. Jones, University College London, London, United Kingdom, and accepted by Editorial Board Member William H. Press December 16, 2020 (received for review August 6, 2020)

April 5, 2021 | 118 (15) e2016239118 | https://doi.org/10.1073/pnas.2016239118

ESM (**Evolutionary Scale Modeling**)

Any questions?



Science

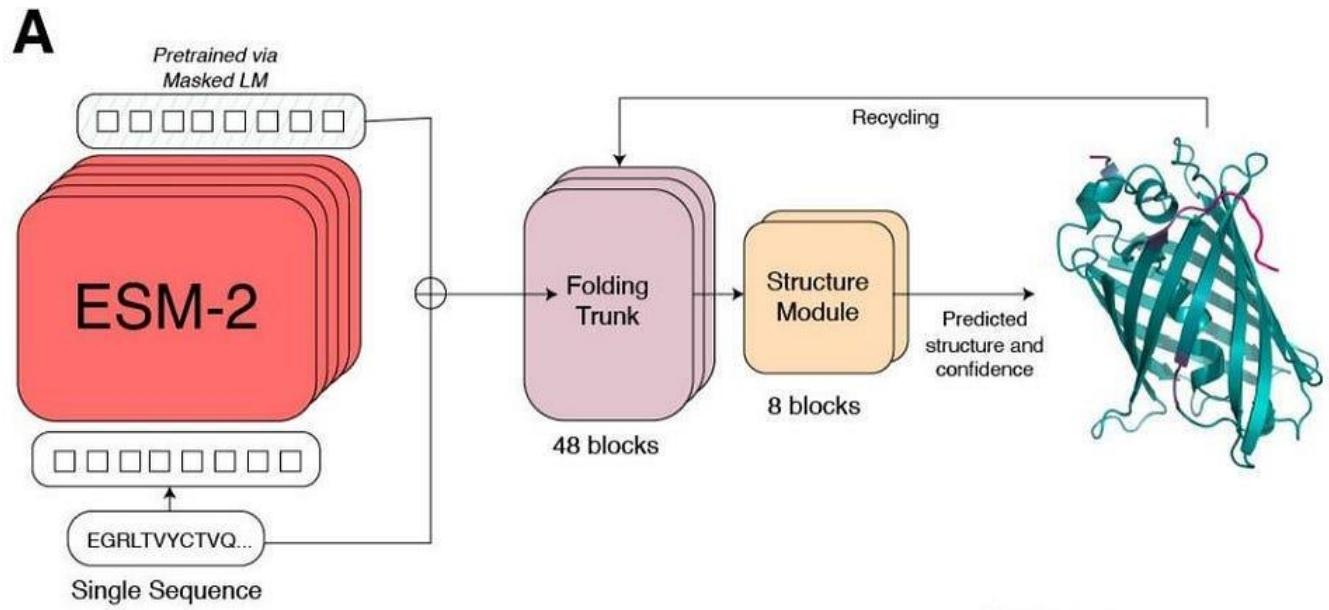
Current Issue First release papers Archive About Submit

HOME > SCIENCE > VOL. 379, NO. 6637 > EVOLUTIONARY-SCALE PREDICTION OF ATOMIC-LEVEL PROTEIN STRUCTURE WITH A LANGUAGE MODEL

RESEARCH ARTICLE | STRUCTURE PREDICTION

Evolutionary-scale prediction of atomic-level protein structure with a language model

ZEMING LIN, HALIL AKIN, ROSHAN RAO, BRIAN HIE, ZHONGKAI ZHU, WENTING LU, NIKITA SMETANIN, ROBERT VERKUIL, ORI KABELI, YANIV SHMUELI, ALLAN DOS SANTOS COSTA, MARYAM FAZEL-ZARANDI, TOM SERCU, SALVATORE CANDIDO, AND ALEXANDER RIVES fewer

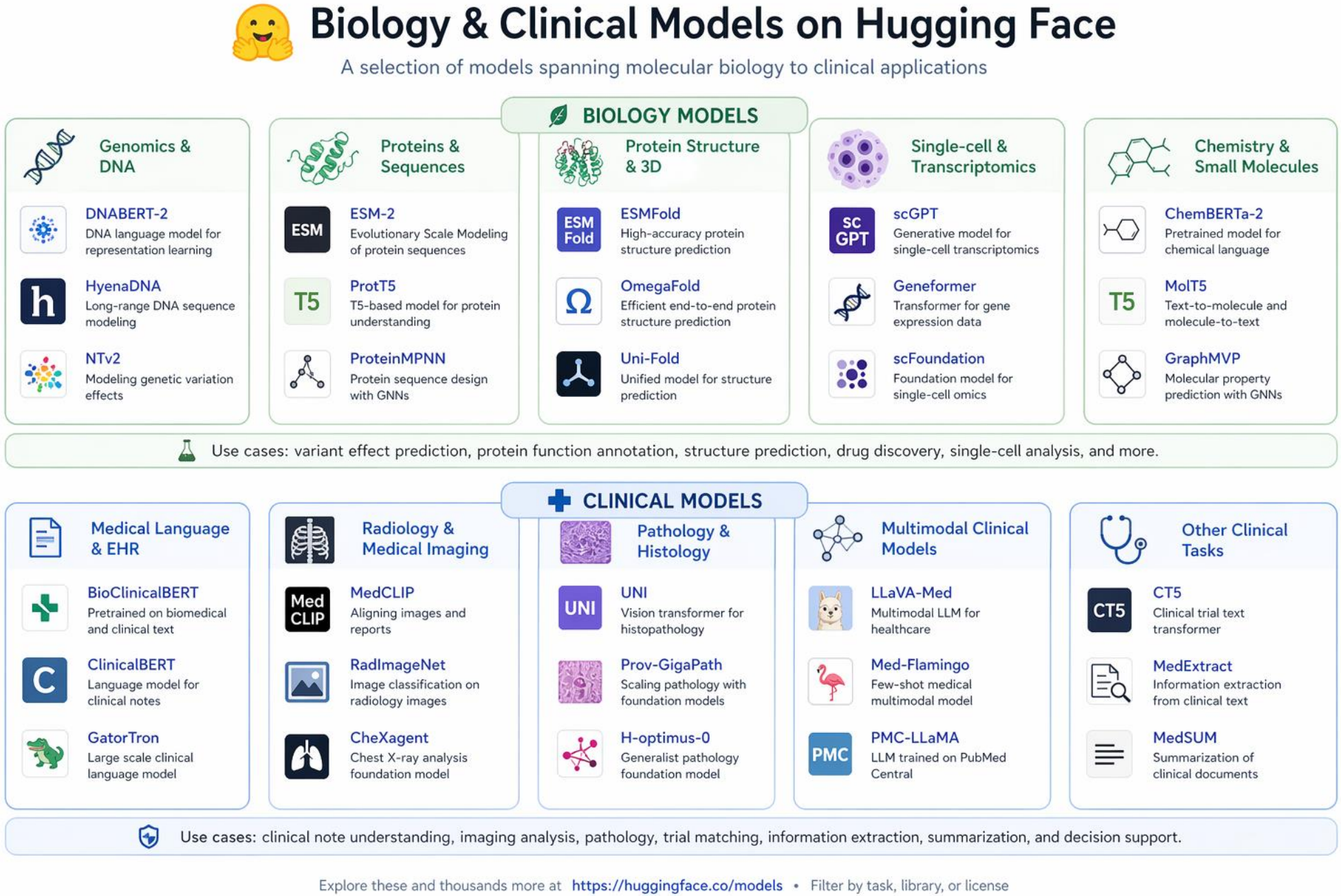


- **ESM-1b** (2020): The original 650M model, proved protein LMs work at scale
- **ESM-1v** (2021): Same architecture as ESM-1b but trained on UniRef90, specialized for variant effect prediction
- **ESM-2** (2022): Next generation, 8M–15B parameter range, rotary embeddings, much better per-parameter performance, enabled ESMFold for structure prediction
- **ESM-3** (2024): 98B parameters, multimodal (sequence + structure + function), generative

Hugging Face

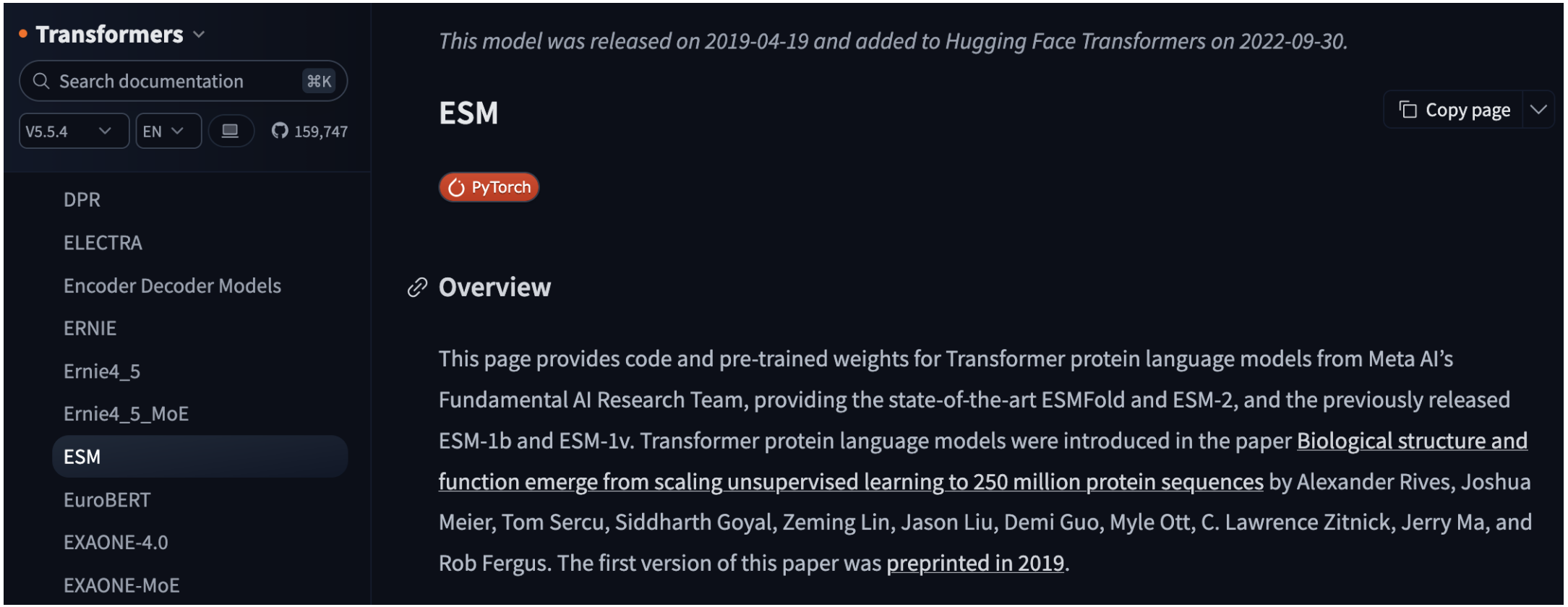
Hugging Face (HF) is *an organization and a platform that provides machine learning models and datasets* with a focus on natural language processing.

Access the lab notebook!



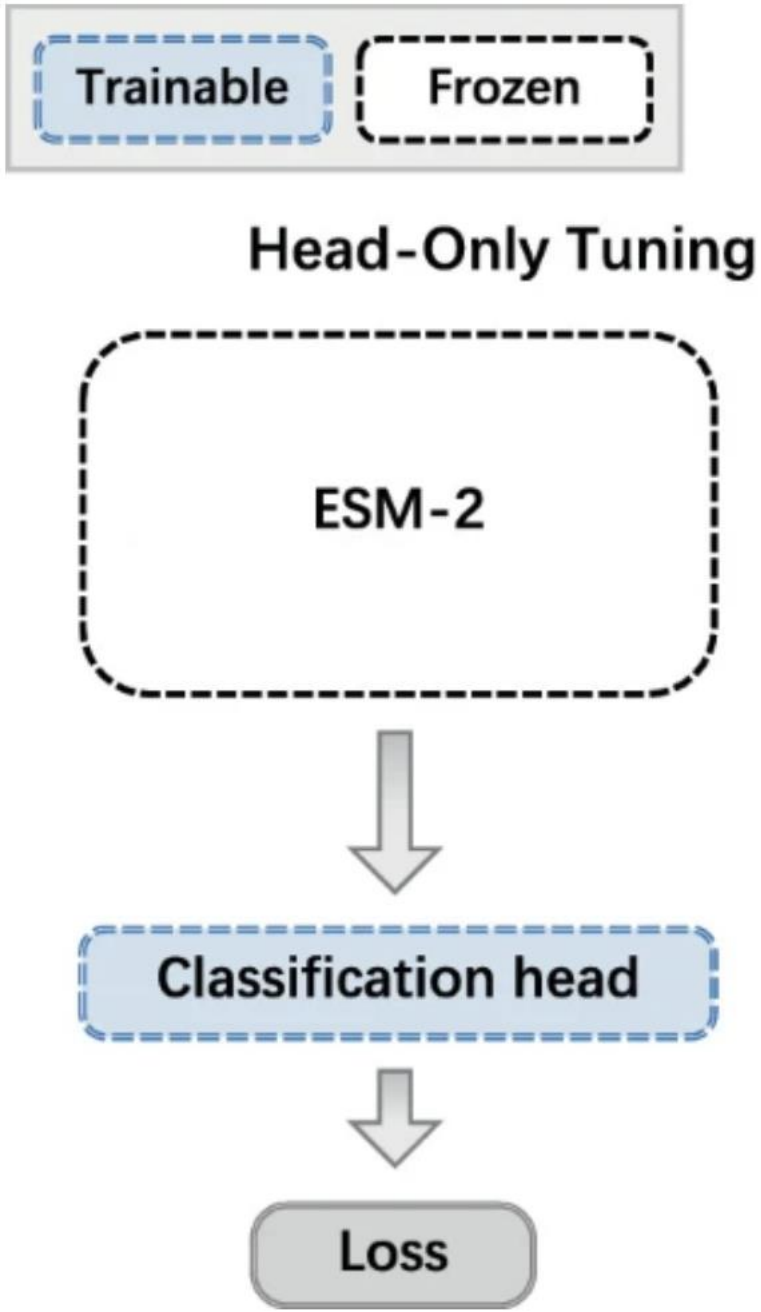
ChatGPT prompt: “generate a figure that shows some of the biology and clinical models on Hugging face”

https://huggingface.co/docs/transformers/model_doc/esm



```
from transformers import AutoTokenizer, AutoModel
import torch

# Load pre-trained ESM-2 model and tokenizer
model_name = "facebook/esm2_t33_650M_UR50D"  # 650M param version
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

model.eval()

# Example protein sequence
sequence = "MKTVRQERLKSIVRILERSKEPVSGAQLAEELSVSRQVIVQDIAYLRSLGYNIVATPRGYVLA"

# Tokenize and get embeddings
inputs = tokenizer(sequence, return_tensors="pt", add_special_tokens=True)
# Input: 1 sequence of 61 residues + 2 special tokens (cls + eos) = 63 tokens

with torch.no_grad():
    outputs = model(input_ids=inputs["input_ids"], attention_mask=inputs["attention_mask"])

# Per-residue embeddings (batch, seq_len, hidden_dim)
residue_embeddings = outputs.last_hidden_state

# Mean-pool over residues for a single sequence-level embedding
# (exclude BOS/EOS tokens at positions 0 and -1)
sequence_embedding = residue_embeddings[0, 1:-1, :].mean(dim=0)

print(f"Residue embeddings shape: {residue_embeddings.shape}")
print(f"Sequence embedding shape: {sequence_embedding.shape}")
```

Suppose you had to do function prediction, what would you do next?



Head-only training means you freeze the pretrained ESM weights and only train a small task-specific layer on top (e.g. a linear layer or small MLP)

Head-only is preferable when:

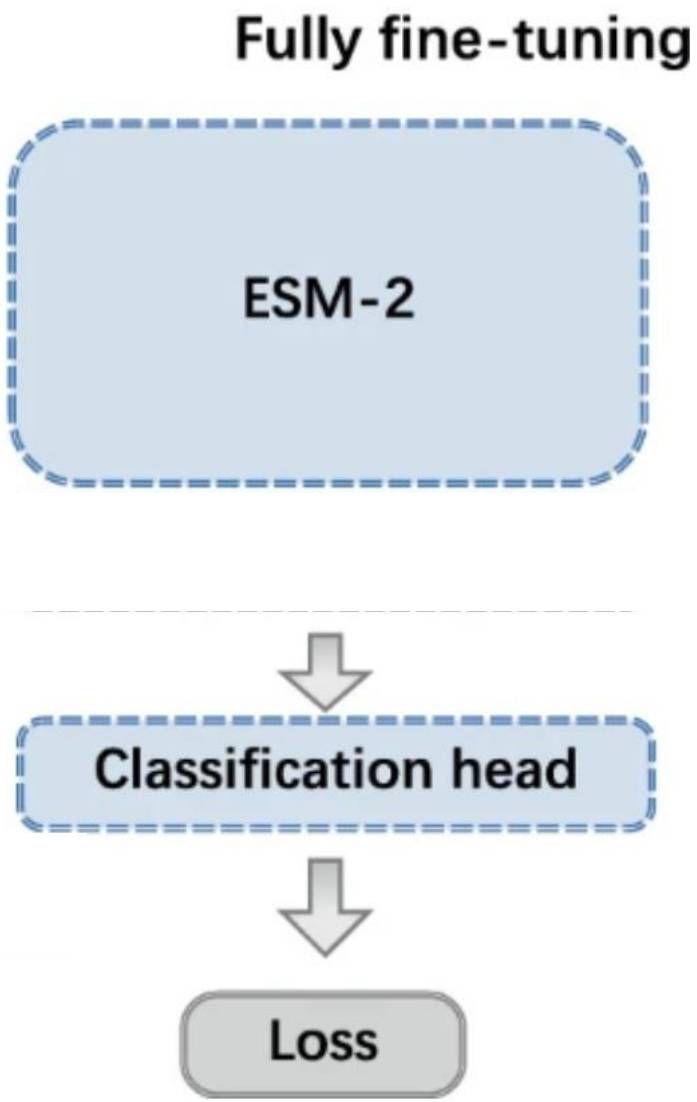
• Have a small labeled dataset (hundreds to low thousands of examples). Updating millions of ESM parameters on little data risks overfitting, while training a small head is much safer.

• Want fast, cheap iteration. Frozen ESM means you compute embeddings once, cache them, and then training the head takes seconds on a CPU.

• **The task aligns well with what ESM already learned.** Things like secondary structure prediction or conservation scoring are "close" to what masked language modeling captures, so the frozen representations are often good enough.

https://link.springer.com/chapter/10.1007/978-981-97-5128-0_30

Fine-tuning



Fine-tuning means you also update the ESM weights themselves.

Fine-tuning is preferable when:

• Have a larger labeled dataset (thousands to tens of thousands+) and can afford the compute.

• The task is "far" from pretraining. For example, predicting a specific experimental property like thermostability or binding affinity to a particular target. The pretrained representations may need adjustment to capture task-specific signal.

• Have plateaued with head-only training and need that extra performance.

• **The sequence distribution in your task is very different from the pretraining data** (e.g. synthetic or heavily engineered sequences).

https://link.springer.com/chapter/10.1007/978-981-97-5128-0_30

In practice, the typical workflow is:

1. Start with head-only training as a baseline

2. If performance is insufficient, try fine-tuning with a low learning rate for the ESM layers (e.g. 1e-5) and a higher rate for the head (e.g. 1e-3)

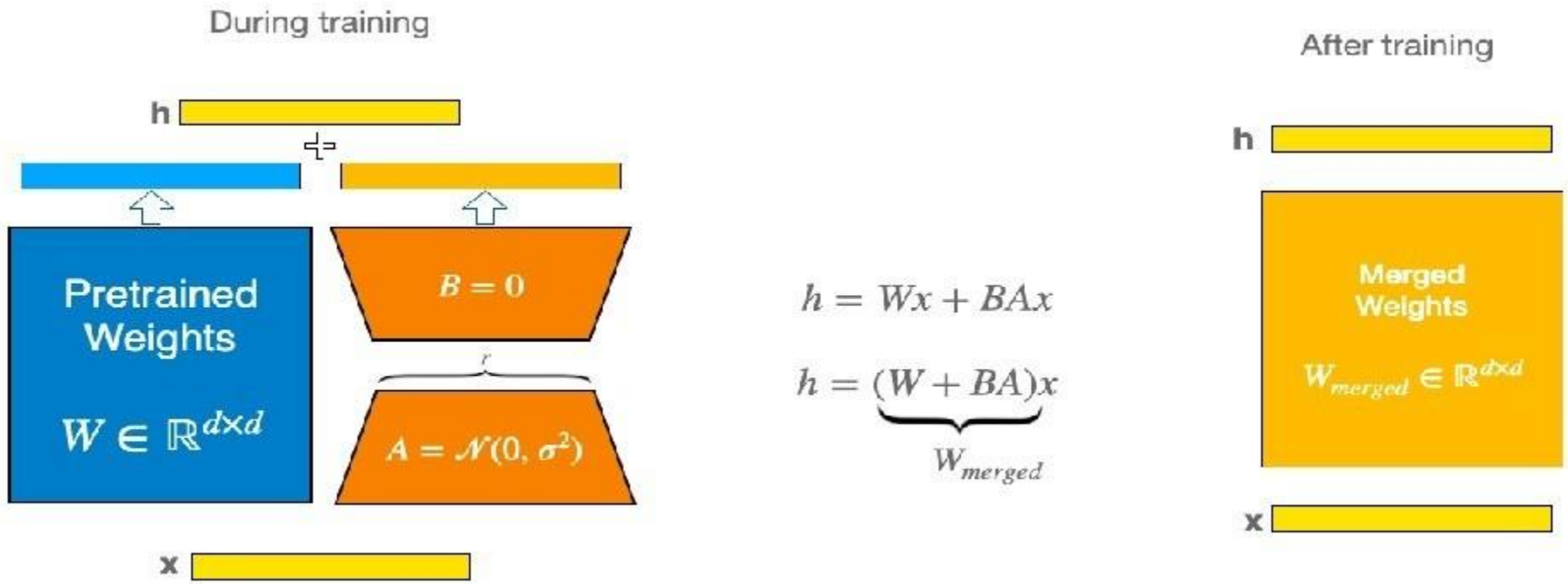
3. **Use LoRA or similar parameter-efficient methods** as a middle ground: you update a small number of adapter parameters inside ESM without modifying the full weights, getting most of fine-tuning's benefit with less overfitting risk and lower memory cost

Any questions?

LoRA Finetuning

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu



Instead of learning a full update ΔW, LoRA factorizes it into two smaller matrices:

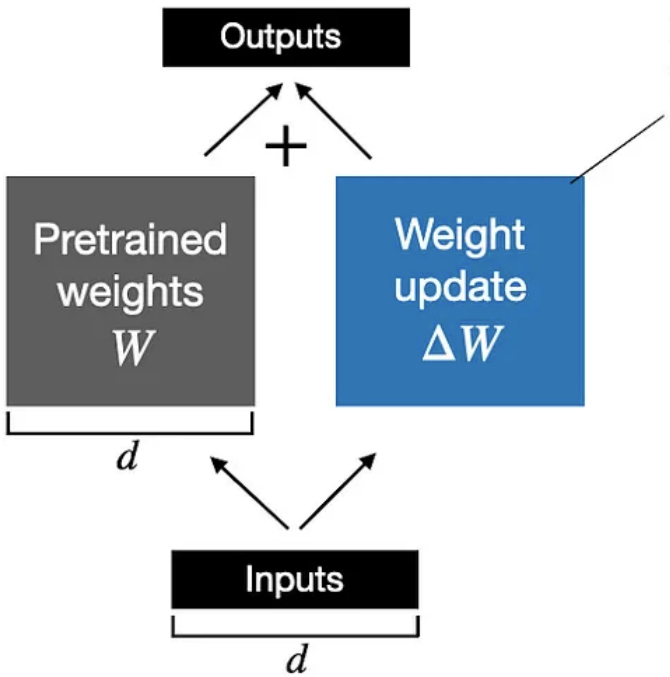
$$\Delta W \approx AB$$

where:

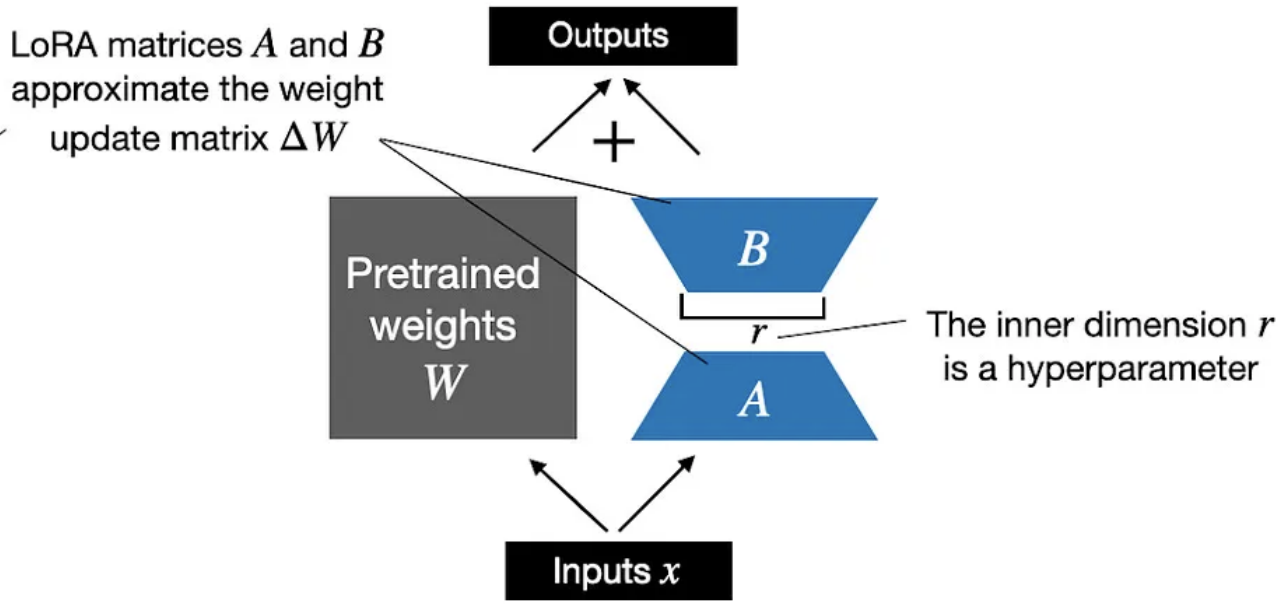
- A is $d \times r$
- B is $r \times k$
- $r \ll d, k$ (this is the "low rank")

The *change* you need to make to W can be approximated by a **low-rank matrix**.

Weight update in regular finetuning



Weight update in LoRA



https://magazine.sebastianraschka.com/p/lora-and-dora-from-scratch

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from peft import LoraConfig, get_peft_model, TaskType
# PEFT: Parameter-Efficient Fine-Tuning
import torch

# Load ESM-2 with a classification head
model_name = "facebook/esm2_t33_650M_UR50D"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)

# Configure LoRA
lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,                            # rank of the low-rank matrices
    lora_alpha=16,                  # scaling factor
    lora_dropout=0.1,               # dropout on LoRA layers
    target_modules=["query", "value"],  # which attention matrices to adapt
)

# Wrap the model with LoRA
model = get_peft_model(model, lora_config)

# See how few parameters you're actually training
model.print_trainable_parameters()
# e.g. "trainable params: 1.8M || all params: 652M || trainable%: 0.28%"
```

```
# Training loop (minimal example)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
sequences = ["MKTVRQERLKSIVRI", "AACDEFGHIKLMNPQ"]
labels = torch.tensor([0, 1])
inputs = tokenizer(sequences, return_tensors="pt", padding=True,
add_special_tokens=True)
inputs["labels"] = labels

model.train()
outputs = model(
    input_ids=inputs["input_ids"],
    attention_mask=inputs["attention_mask"],
    labels=inputs["labels"],)

loss = outputs.loss
loss.backward()
optimizer.step()
optimizer.zero_grad()
print(f"Loss: {loss.item():.4f}")

# Save and reload the LoRA adapter (only saves the small adapter weights)
model.save_pretrained("esm2_lora_adapter")
# Later: model = AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=2)
#        model = PeftModel.from_pretrained(model, "esm2_lora_adapter")
```

Recap– learn about protein language models and how to use them

(1) Protein Language Models

(2) Hugging Face

(3) LoRA fine-tuning

(3) Class activity: ESM model for protein sequences

Wrap up

What was the clearest point today?

What was the muddiest point today?